

```
###
# セットアップ
###

import sys
import os
from pathlib import Path

from google.colab import drive
drive.mount("/content/drive")

ROOT_PATH = Path("/content/drive/MyDrive/cnn-hands-on")

if str(ROOT_PATH) not in sys.path:
    sys.path.append(str(ROOT_PATH))
```

```
###
# Cell 01
###

import torch
import torchvision.transforms as transforms
from PIL import Image
import pandas as pd
import matplotlib.pyplot as plt
import os
import random
from google.colab import files

from utils.network import SimpleCNN # pyright: ignore[reportMissingImports]
from utils.dataloader import get_cat_dog_data loaders # pyright: ignore[reportMis
```

```
###
# Cell 02
###

model = SimpleCNN()

model_path = ROOT_PATH / "models" / "01_intro.pth"

model.load_state_dict(torch.load(model_path, map_location=torch.device("cpu")))

model.eval()

categories = ["Cat(猫)", "Dog(犬)"]
```

```
###
# Cell 03
###

_, test_loader = get_cat_dog_data loaders(
    root_path=ROOT_PATH,
    data_size="small",
    batch_size=1
)

test_dataset = test_loader.dataset

print(f"{len(test_dataset)}枚の画像が準備完了")
```

```
###
# Cell 04
###

random_idx = random.randint(0, len(test_dataset) - 1)
input_tensor, true_label_idx = test_dataset[random_idx]
true_class = categories[true_label_idx]

image_for_show = input_tensor.permute(1,2,0)
plt.imshow(image_for_show)
plt.axis("off")
plt.show()

input_batch = input_tensor.unsqueeze(0)

with torch.no_grad():
    output = model(input_batch)

prob = torch.nn.functional.softmax(output[0], dim=0)
pred_idx = torch.argmax(prob).item()
pred_class = categories[pred_idx]

print(f"予想: {pred_class}, 正解: {true_class}")
```

```
###
# Cell 05
###

preprocess = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor(),
])

print("判定したい画像ファイルを選んでください")
uploaded = files.upload()

if len(uploaded) > 0:
    filename = next(iter(uploaded))

    my_image = Image.open(filename).convert("RGB")
    plt.imshow(my_image)
    plt.axis("off")
    plt.show()

    my_tensor = preprocess(my_image)
    my_batch = my_tensor.unsqueeze(0)

    with torch.no_grad():
        output = model(my_batch)

    prob = torch.nn.functional.softmax(output[0], dim=0)
    best_idx = torch.argmax(prob).item()
    best_score = prob[best_idx].item() * 100
    class_name = categories[best_idx]

    print(f"予想: {class_name}")
else:
    print("画像のアップデートができませんでした")
```